

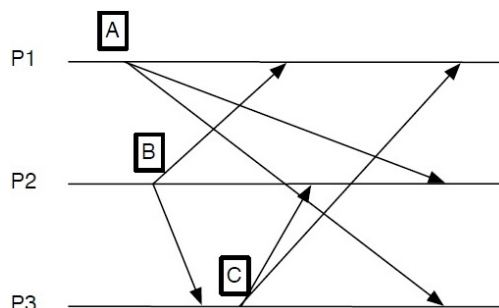
Sistemes Distribuïts en Xarxa (SDX)
Facultat d'Informàtica de Barcelona
Control parcial. 26 d'Abril 2023

Contesteu a les preguntes de manera concisa i precisa
Contesteu al mateix full
Test, seminaris: no es poden consultar apunts
Readings: podeu portar els vostres informes
Problemes: podeu portar un full de notes imprès a doble cara
Durada: 1 hora, 50 minuts

Nom i Cognoms:

1. (5 pts) Seleccioneu la resposta (una només) que considereu correcta en cadascun dels apartats. Cada resposta correcta val 1/2 punts. Cada resposta incorrecta resta 1/6.
 - (a) Quina de les següents NO és una característica dels sistemes distribuïts?
 - ☐ Els nodes es comuniquen enviant missatges per la xarxa, i això introdueix un retard en la comunicació
 - ☐ Els nodes poden fallar de manera independent, quedant afectats només alguns d'ells i permetent que els altres puguin continuar funcionant
 - ☐ Els nodes no sempre estan disponibles degut a particions de xarxa o desconnexions
 - ✓ Cada node té un rellotge independent, però poden estar perfectament sincronitzats
 - (b) Quin dels següents tipus de fallada es produeix quan no podem editar un document amb Google Docs des del nostre telèfon mòbil per manca de cobertura?
 - ☐ Timing failure
 - ☐ Arbitrary (Byzantine) failure
 - ✓ Omission failure
 - ☐ Crash failure
 - (c) En quin dels següents tipus de comunicació l'emissor i el receptor no han de conèixer les seves identitats per poder comunicar-se?
 - ✓ Space-decoupled communication
 - ☐ Time-decoupled communication
 - ☐ Direct communication
 - ☐ Asynchronous communication
 - (d) Tenim un conjunt de quatre nodes que volen sincronitzar el seu rellotge mitjançant l'algorisme de *Berkeley*. El temps actual del node màster és 3:52:21.000. Aquest envia peticions simultànies als altres nodes, i les seves respostes inclouen els temps 3:52:20.150, 3:52:20.800 i 3:52:20.750, respectivament. El temps de *round-trip* d'aquestes peticions ha estat 50, 50 i 100 ms, respectivament. Si la màxima desviació dels rellotges permesa en aquest sistema és 0:00:00.500, a quin valor fixarà cada node el seu rellotge?
 - ☐ 3:52:20.675
 - ☐ 3:52:20.700
 - ☐ 3:52:20.850
 - ✓ 3:52:20.875

- (e) Quina de les següents operacions és idempotent?
- ☐ Escriure un byte al final d'un fitxer en un sistema de fitxers Linux
 - ☐ Escriure un byte en la posició actual d'un fitxer en un sistema de fitxers Linux
 - ✓ ☒ Llegir un byte del principi d'un fitxer en un sistema de fitxers Linux
 - ☐ Llegir un byte de la posició actual d'un fitxer en un sistema de fitxers Linux
- (f) Quin dels següents protocols de *routing* d'events en sistemes de publicació/subscripció realitza una notificació d'events més ràpida però guardant una quantitat més gran d'informació en els nodes?
- ☐ Event flooding routing
 - ✓ ☒ Subscription flooding routing
 - ☐ Gossip-based routing
 - ☐ Filter-based routing
- (g) Tenim un grup de sis processos amb identificadors P1 fins P6. El líder actual P6 té una fallada de crash i P2 inicia una elecció mitjançant l'algorisme *Bully*. Si la latència entre qualsevol parell de processos és de 10 ms, quant temps trigarà en finalitzar l'elecció? (pots assumir que no hi ha fallades, una única elecció en curs, i temps de processament dels missatges negligible)
- ☐ 20 ms
 - ☐ 30 ms
 - ✓ ☒ 40 ms
 - ☐ 50 ms
- (h) Tenim un grup de sis processos amb identificadors P1 fins P6 organitzats en sentit horari en un anell lògic. Si P4 inicia una elecció, quants missatges són necessaris per escollir un líder? (pots assumir que no hi ha fallades i una única elecció en curs)
- ☐ 11 missatges, si es fa servir l'algorisme *Bully*
 - ☐ 14 missatges, si es fa servir l'algorisme de *Chang & Roberts*
 - ☐ 12 missatges, si es fa servir l'algorisme en anell millorat (i.e., *Enhanced Ring algorithm*)
 - ✓ ☒ Totes les anteriors són certes
- (i) Tenim un grup de processos que es comuniquen mitjançant la variant de multicast totalment ordenat en què els processos decideixen de manera coordinada els números de seqüència dels missatges. La figura mostra l'enviament i recepció del primer missatge de l'algorisme de coordinació pels missatges multicast *A*, *B* i *C* enviats pels processos P1, P2 i P3, respectivament. En quin ordre total es lliuraran al nivell aplicació aquests tres missatges?



✓ ☒ B - C - A

☐ A - B - C

☐ B - A - C

☐ C - A - B

- (j) Donat un *data store* eventualment consistent que usa un mecanisme de *version vectors* per la resolució de conflictes d'escriptura sobre un objecte que actualment té un únic valor, en quina situació es guardarà el valor d'una nova operació d'escriptura com un *sibling*?
- ☐ Mai, la nova escriptura només pot sobreescriure el valor actual o ser descartada
 - ✓ ☒ En cas que el valor actual hagués estat modificat per una escriptura concurrent a la nova operació d'escriptura
 - ☐ En cas que el valor actual hagués estat modificat per una escriptura anterior (segons l'ordenació *happened-before*) a la nova operació d'escriptura
 - ☐ Sempre

Nom i Cognoms:

2. (SEMINARIS) Contesta les següents preguntes referents als seminaris **Muty** i **Chatty**.

- a) (6 pts) **Muty**. Modifica el següent extracte de codi per tal que es comporti conforme a la implementació de l'exclusió mútua de Ricart & Agrawala amb prioritats (mòdul *lock2*).

```
wait(Nodes, Master, Refs, Waiting) ->
  receive
    {request, From, Ref} ->
      wait(Nodes, Master, Refs, [{From, Ref}|Waiting]);
    {ok, Ref} ->
      NewRefs = lists:delete(Ref, Refs),
      wait(Nodes, Master, NewRefs, Waiting)
  end.

% MyId: Priority of this process
% ReqId: Priority of the process sending the 'request' message
wait(Nodes, Master, Refs, Waiting, MyId) ->
  receive
    {request, From, Ref, ReqId} ->
      if
        ReqId < MyId ->
          From ! {ok, Ref},
          R = make_ref(),
          From ! {request, self(), R, MyId},
          wait(Nodes, Master, [R|Refs], Waiting, MyId);
        true ->
          wait(Nodes, Master, Refs, [{From, Ref}|Waiting], MyId)
      end;
    {ok, Ref} ->
      NewRefs = lists:delete(Ref, Refs),
      wait(Nodes, Master, NewRefs, Waiting, MyId)
  end.
```

- b) (2 pts) **Muty**. En la implementació del *lock3*, és possible que un worker accedeixi a la regió crítica abans que un altre worker que ho va demanar a la seva instància de lock amb anterioritat (segons un criteri d'ordenació *happened-before*)? Justifica la resposta amb un exemple.

Solution: Yes, it is possible. For example, worker *w1* decides to enter the critical section and sends a **take** message to its lock instance *l1*. It also sends a notification message to worker *w2*. When receiving that message, *w2* also decides to enter the critical section (thus, the request by *w1* happened-before) and sends the corresponding **take** message to its lock instance *l2*. As **take** messages do not include the logical clock, there is no way to determine their happened-before ordering. The order to access the critical section will be determined according to the logical time of each lock instance when it receives the **take** message and sends the **request** messages. If the logical time of lock instance *l2* is lower than the time of *l1* (or the times are equal and the ID of *w2* is lower), worker *w2* will be granted access first, despite the request by *w1* happened-before.

- c) (2 pts) **Chatty**. Justifica com és possible que un client que abandona el xat mentre el servidor està enviant un missatge multicast rebi aquest missatge i l'escrigui per pantalla.

Solution: If a client leaves during a multicast, he/she will receive the message, as he/she is still in the server's list of clients and the client's procedure **process_requests** keeps listening for messages from the server until an 'exit' message is received.

3. (SEMINARIS) Contesta les següents preguntes referents als seminaris **Ordy** i **Chatty**.

- a) (8 pts) **Ordy**. Completa el següent extracte de codi corresponent a la implementació de multicast totalment ordenat.

```
server(Master, MaxPrp, MaxAgr, Nodes, Cast, Queue) ->
  receive
    {send, Msg} ->
      Ref = make_ref(),
      Self = self(),
      lists:foreach(fun(Node) ->
        Node ! {request, Self, Ref, Msg}
      end, Nodes),
      NewCast = [{Ref, length(Nodes), seq:null()}|Cast],
      server(Master, MaxPrp, MaxAgr, Nodes, NewCast, Queue);
    {request, From, Ref, Msg} ->
      NewMaxPrp = seq:maxIncrement(MaxPrp, MaxAgr),
      From ! {proposal, Ref, NewMaxPrp},
      NewQueue = queue(Ref, Msg, prospd, NewMaxPrp, Queue),
      server(Master, NewMaxPrp, MaxAgr, Nodes, Cast, NewQueue);
    {proposal, Ref, Num} ->
      case proposal(Ref, Num, Cast) of
        {agreed, MaxNum, NewCast} ->
          lists:foreach(fun(Node)->
            Node ! {agreed, Ref, MaxNum}
          end, Nodes),
          server(Master, MaxPrp, MaxAgr, Nodes, NewCast, Queue);
        NewCast ->
          server(Master, MaxPrp, MaxAgr, Nodes, NewCast, Queue)
      end;
    {agreed, Ref, Num} ->
      NewQueue = update(Ref, Num, Queue),
      {AgrMsg, NewerQueue} = agreed(NewQueue),
      lists:foreach(fun(Msg)->
        Master ! {deliver, Msg}
      end, AgrMsg),
      NewMaxAgr = seq:max(MaxAgr, Num),
      server(Master, MaxPrp, NewMaxAgr, Nodes, Cast, NewerQueue)
  end.
```

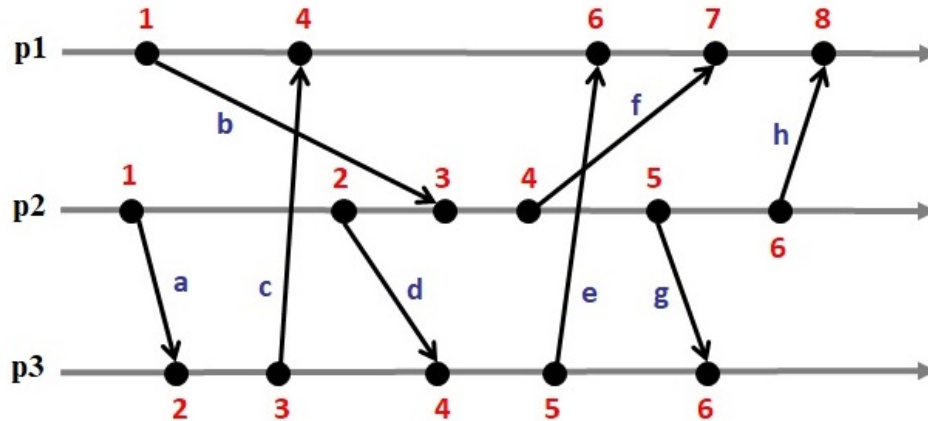
- b) (2 pts) **Chatty**. És possible que un client rebi un missatge m_2 , el qual és una resposta d'un altre client a un missatge m_1 enviat per un tercer client, abans de rebre el missatge m_1 ? Justifica la resposta en funció de si tots els clients estan connectats a un únic servidor o cadascun a un servidor diferent.

Solution: It is not possible when clients are connected to a single server, because Erlang guarantees FIFO ordering between every client and the server and, therefore, m_2 cannot overtake m_1 for any client. It is possible when clients are connected to different servers, because m_2 could reach the server which this client is connected to before m_1 , as they come from different servers.

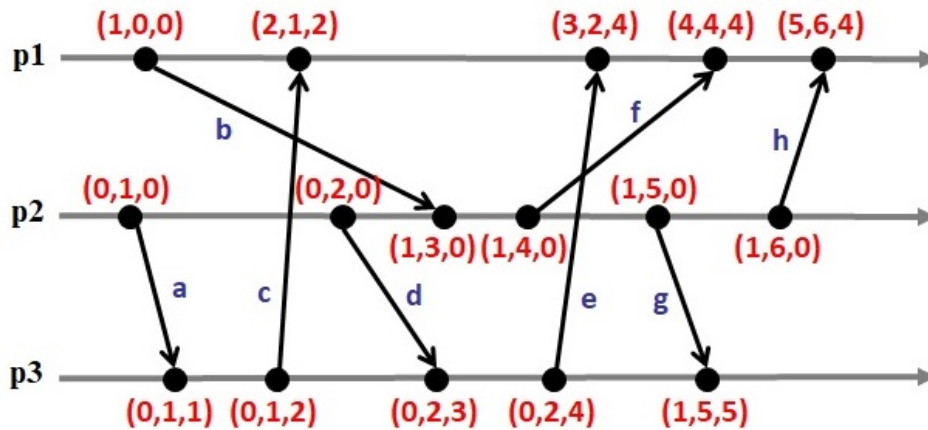
Nom i Cognoms:

4. (1.25 pts) Donada la següent seqüència d'events executada pels processos $p1$, $p2$ i $p3$.

a) Etiqueta cada event amb el valor del seu rellotge lògic escalar (i.e., rellotge de Lamport).

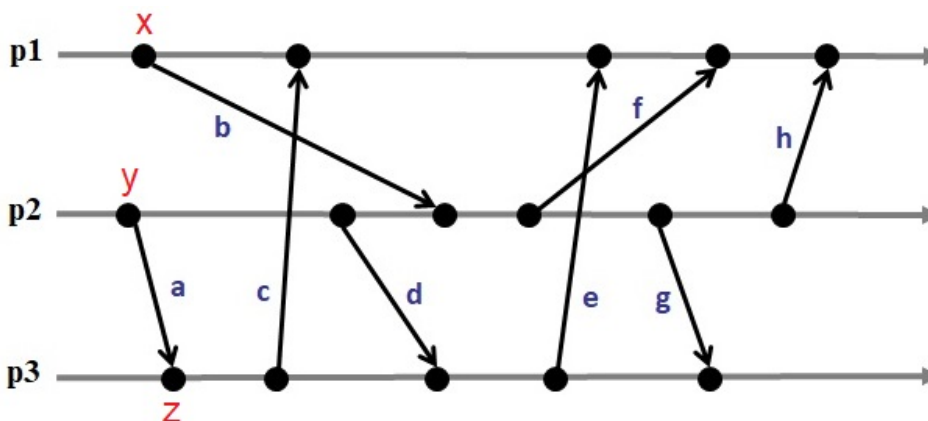


b) Etiqueta cada event amb el valor del seu rellotge lògic vectorial.



c) Donades les notacions $x \rightarrow y$ per indicar que x ha passat abans que y , i $x \parallel y$ per indicar que x i y són concurrents, algú ens assegura que si $x \parallel y$ i $y \rightarrow z$, llavors $x \rightarrow z$. Troba un contraexemple en l'apartat anterior per demostrar que s'equivoca.

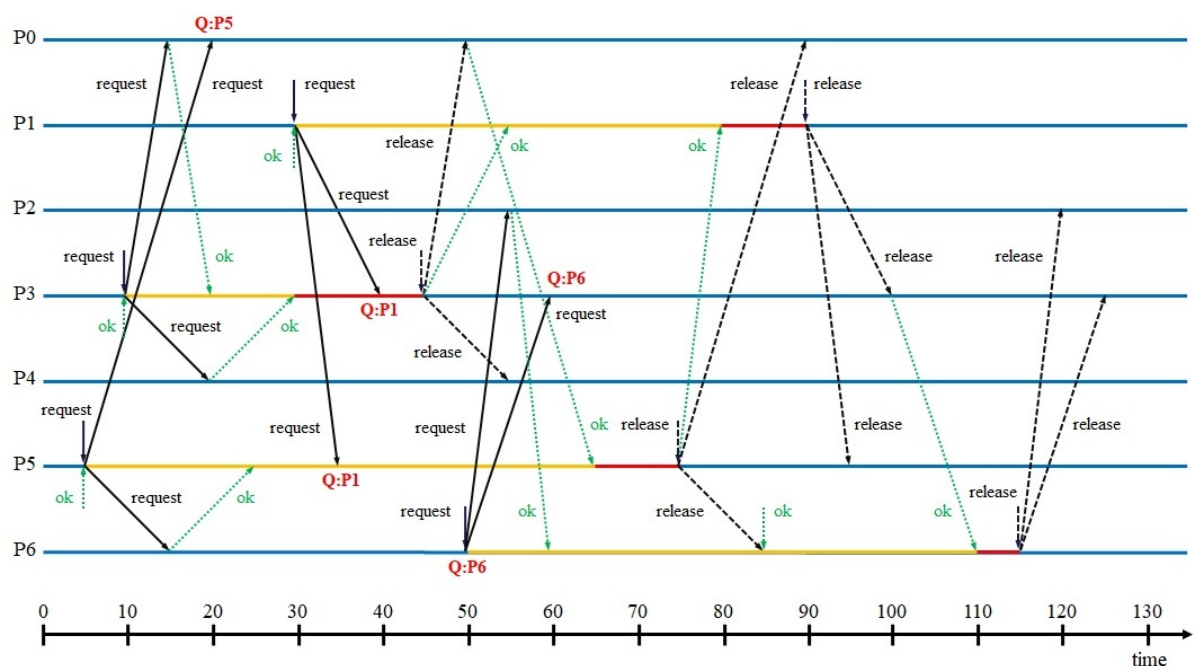
Solution: $x \parallel y$ and $y \rightarrow z$ but $x \parallel z$



5. (1.25 pts) Tenim un conjunt de set processos ($P0-P6$) que coordinen els seus accessos a una regió crítica mitjançant l'algorisme de *Maekawa*, el qual està configurat amb els següents subconjunts de votació: $S0 = \{0, 1, 2\}$, $S1 = \{1, 3, 5\}$, $S2 = \{2, 4, 5\}$, $S3 = \{0, 3, 4\}$, $S4 = \{1, 4, 6\}$, $S5 = \{0, 5, 6\}$, $S6 = \{2, 3, 6\}$. $P1$, $P3$, $P5$ i $P6$ volen accedir a la regió crítica de manera concurrent. La taula de l'esquerra detalla per cada procés en quin instant de temps demana accedir a la regió crítica (columna 1) i quant temps treballa dins de la regió crítica abans d'alliberar-la (columna 2). La taula de la dreta especifica les latències de comunicació entre qualsevol parell de processos (són simètriques).

	temps petició	temps dins regió crítica
$P1$	30	10
$P3$	10	15
$P5$	5	10
$P6$	50	5

	$P0$	$P1$	$P2$	$P3$	$P4$	$P5$	$P6$
$P0$	0	20	20	5	20	15	20
$P1$	20	0	20	10	20	5	20
$P2$	20	20	0	20	20	20	5
$P3$	5	10	20	0	10	20	10
$P4$	20	20	20	10	0	20	20
$P5$	15	5	20	20	20	0	10
$P6$	20	20	5	10	20	10	0



- a) Ajuda't del cronograma per calcular en quin instant accedirà cada procés a la regió crítica.

Solution:

$P1$ accesses the critical section at $t = 80$.

$P3$ accesses the critical section at $t = 30$.

$P5$ accesses the critical section at $t = 65$.

$P6$ accesses the critical section at $t = 110$.

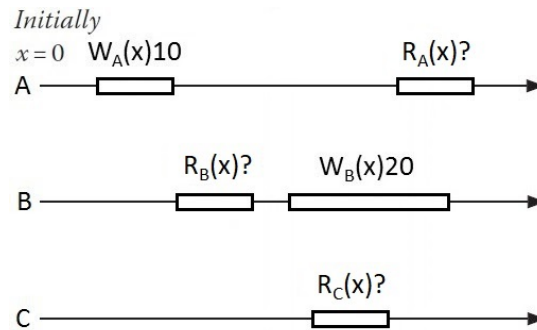
- b) Calcula el nombre de missatges necessaris perquè un procés pugui entrar i sortir de la regió crítica en l'escenari previ, i compara'l amb l'algorisme de *Ricart and Agrawala*.

Solution: *Maekawa:* Let K be the size of the subsets, each process sends/receives $3K$ messages (K request + K OK + K release). In this example, this equals 9 messages.

Ricart and Agrawala: Let N be the total number of processes, each process sends/receives $2(N - 1)$ messages ($N - 1$ request + $N - 1$ OK). In this example, this equals 12 messages.

Nom i Cognoms:

6. (1.25 pts) Tenim tres clients A, B i C que executen les operacions tal com es mostren a la figura següent en un *data store* de 11 rèpliques que utilitza un protocol d'escriptura replicada (i.e., *replicated-write protocol*) basat en replicació activa (i.e., *active replication*) i que proporciona el model de consistència de *linearizability*.



- a) Explica com es duran a terme les operacions $W_B(x)20$ i $R_C(x)?$ (quines rèpliques es contacten, amb quin tipus de missatge, com es determina el resultat, quan es retorna el control al client).

Solution: $W_B(x)20$: Client B sends the write request to all the replicas using total+FIFO-ordered multicast. The write operation finishes once the client is delivered its own message. $R_C(x)?$: Client C sends the read request to all the replicas using total+FIFO-ordered multicast. The read operation waits until a number of replies has been collected. The specific number depends upon the failure model. If only crash failures are supported, the first response to arrive back is returned to the client. If Byzantine failures are supported, a majority of identical responses is needed.

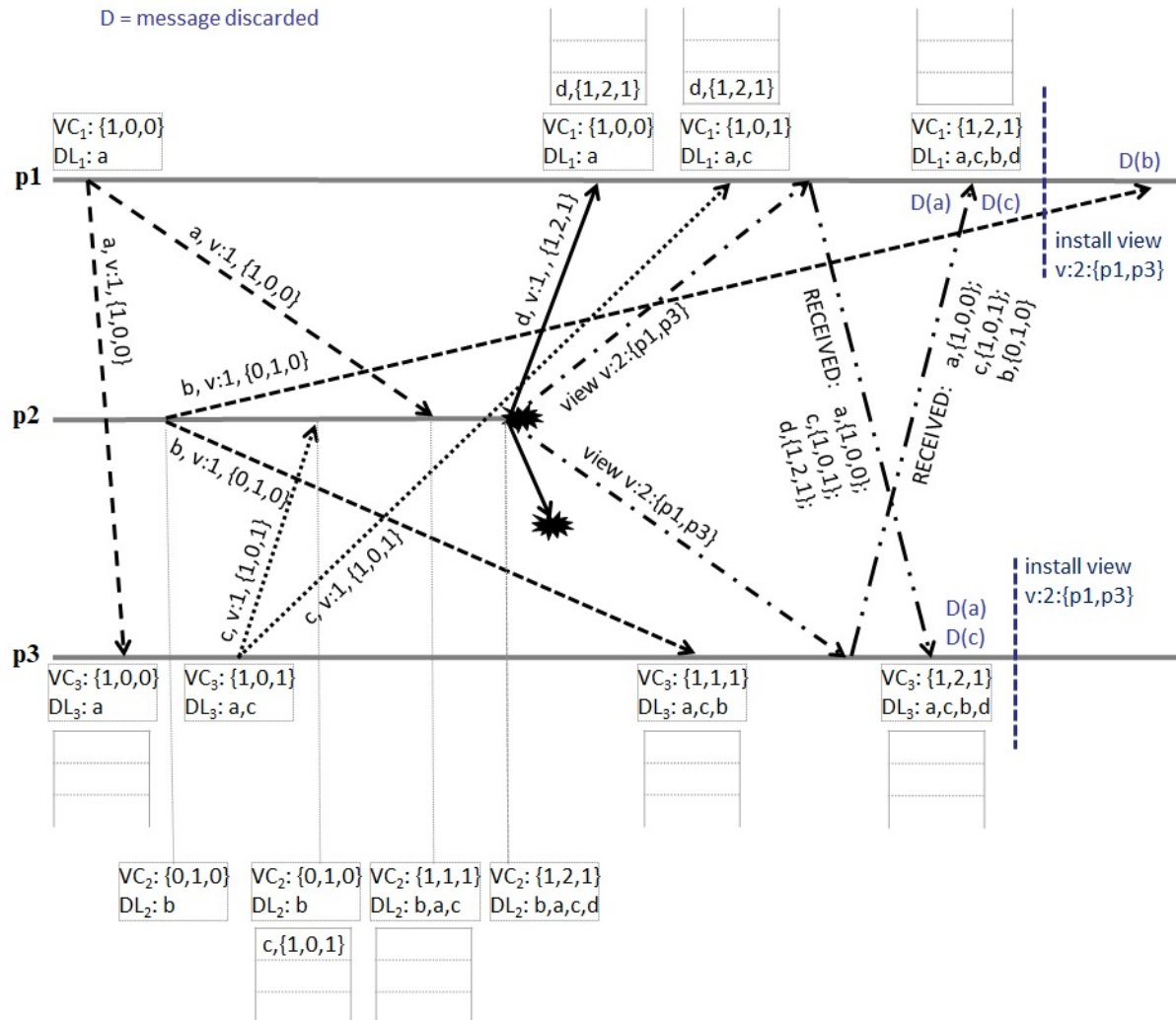
- b) Indica quin possibles valors poden retornar les operacions $R_A(x)?$, $R_B(x)?$ i $R_C(x)?$

Solution: $R_B(x)?$ can only return 10, which is the value written by the most recent non-overlapping write operation, as it does not overlap with any write operation. $R_A(x)?$ and $R_C(x)?$ overlap with $W_B(x)20$, hence, they may return 10 (i.e., the value written by the most recent non-overlapping write operation) or 20, but as $R_A(x)?$ starts after the termination of $R_C(x)?$, then $R_A(x)?$ must read the same or a newer value. In particular, if $R_C(x)?$ returns 10, then $R_A(x)?$ can return 10 or 20, but if $R_C(x)?$ returns 20, then $R_A(x)?$ must also return 20.

- c) Calcula quantes fallades de crash i quantes fallades Byzantines pot suportar aquest *data store*.

Solution: Can tolerate up to 10 crash failures: $K + 1 \leq 11$; $K \leq 10$.
Can tolerate up to 5 Byzantine failures: $2K + 1 \leq 11$; $K \leq (11 - 1)/2$; $K \leq 5$.

7. (1.25 pts) Tenim el següent grup de processos que es comuniquen mitjançant multicast causalment ordenat basat en vistes (i.e., *causally-ordered view-synchronous multicast*), en què la vista actual té identificador $v = 1$ i membres $\{p1, p2, p3\}$. Completa la següent figura indicant per cada procés i : el seu vector clock (VC_i), els missatges guardats a la *hold-back queue* pendents de ser lliurats, i la llista de missatges lliurats fins el moment (DL_i), a més del vector clock dels missatges enviats. Per implementar el canvi de vista, cada procés ha de reenviar tots els missatges que ha rebut durant la vista actual (inclosos els que es troben encara a la *hold-back queue*) als altres processos. Completa l'etiqueta 'RECEIVED:' amb la informació d'aquests missatges, i indica l'estat de cada procés receptor després de rebre'ls (recorda que els missatges duplicats s'han de descartar). Finalment, indica també amb una línia vertical quan es pot instal·lar la nova vista en cada procés.



Nom i Cognoms:

8. (READINGS) Contesta les següents preguntes referents als articles llegits a l'assignatura.

- i) Explica el rol d'Erlang respecte l'assumpció 'The Network Is Secure' i quines mesures poden prendre els desenvolupadors en aquest sentit segons es descriu a l'article *Hebert13*.

Solution: Erlang assumes that the network is secure. Dealing with insecure networks is left to the developers, by either switching to SSL, implementing their own high-level communication layer, tunneling over secure channels, or reimplementing the carrier communication protocol between nodes.

- ii) Indica a quin dels protocols per la sincronització de rellotges descrits a l'article *Neville-Neil16* (i.e., NTP o PTP) es refereix cadascuna de les afirmacions següents:

- Usa un protocol multicast en què el màster envia periòdicament el seu temps als nodes esclaus: **PTP**
- Està implementat com un sistema distribuït que té rellotges de diferents nivells de precisió formant una jerarquia en què els rellotges amb un nombre d'estrat inferior tenen més precisió: **NTP**
- Usa un mecanisme d'enquesta per obtenir els temps d'un conjunt de rellotges que es fan servir per ajustar el rellotge local: **NTP**
- Està dissenyat per conjunts petits de nodes connectats a una xarxa única sense commutadors ni encaminadors: **PTP**

- iii) Explica en què consisteix el model de consistència *Bounded Staleness* tal com es descriu a l'article *Terry13*, i posa un exemple en el context del marcador d'un partit de beisbol.

Solution: Bounded Staleness guarantees that a read operation will return any values written at most T minutes ago. For instance, in a baseball scoreboard, a read operation satisfying this model might return any of the scores which are at most one inning out-of-date.